

The Maths, Decoded

Every equation, with real numbers

15 July 2026

Contents

1 The maths in statement.md, decoded	1
1.1 0. First: the symbols that are not really maths	1
2 PART A — the Wi-Fi equations	2
2.1 A.1 Distance	2
2.2 A.2 Signal strength — the important one	2
2.3 A.3 The S-curve (soft coverage)	3
2.4 A.4 Weighted coverage — the score	3
2.5 A.5 The objective (what we maximise)	4
2.6 A.6 The PSO update	4
3 PART B — the water-pump equations	4
3.1 B.1 The state	4
3.2 B.2 One hour of physics	5
3.3 B.3 The reward	5
3.4 B.4 The subtle one: $R(s, a) = \mathbb{E}_w[r]$	5
3.5 B.5 Discounting — what γ actually does	5
3.6 B.6 The Bellman optimality equation	6
3.7 B.7 The Q-learning update	6
3.8 B.8 Regret	7
3.9 The three sentences to take away	7

1 The maths in statement.md, decoded

statement.md is written the way a professor expects to read it: compressed, symbolic, no hand-holding. That is correct for a report and useless for learning from. This file is the other half. Same equations, but every symbol named, and every formula run with real numbers from our own hall so you can watch it work.

Read this with statement.md open beside it.

1.1 0. First: the symbols that are not really maths

Most of what makes a formula look scary is notation, not ideas. Here is the whole vocabulary used in statement.md.

Symbol	Say it out loud as	Example
$\sum_i x_i$	"add up all the x_i "	$\sum$ of 2, 5, 3 is 10
$\max(a, b)$	"whichever is bigger"	$\max(0, -4) = 0$
$\max_j \text{RSSI}_{ij}$	"the best one, over all the j 's"	best signal room i can hear
$\arg \max_a$	" which a gives the biggest value" (not the value — the <i>choice</i>)	which action is best

Symbol	Say it out loud as	Example
$\ \mathbf{a} - \mathbf{b}\ $	"the straight-line distance from a to b"	Pythagoras
$x \in S$	"x is one of the things in S"	$3 \in \{1, 2, 3\}$
$\mathbb{R}^6$	"a list of 6 ordinary numbers"	our 3 APs, x and y each
$\mathbb{E}[X]$	"the average value of X"	expected value
$\mathbb{1}[\dots]$	"1 if that's true, 0 if not"	a switch
$\sigma(z)$	"squash z into a number between 0 and 1"	the S-curve, below
γ (gamma)	"how much I care about the future"	0.99
π (pi)	not 3.14 here — it means "policy", a rule	$\pi(s)$ = what to do in state s

That table is genuinely most of it. If a formula still looks bad, it is usually because several of these are stacked, not because it is deep.

2 PART A — the Wi-Fi equations

2.1 A.1 Distance

$$d_{ij} = \max(\|\mathbf{r}_i - \mathbf{a}_j\|, 1)$$

In words: how far room i is from access point j , in metres — but never let it be less than 1.

- $\mathbf{r}_i$ = where room i is. $\mathbf{a}_j$ = where AP j is.
- $\|\cdot\|$ = ordinary straight-line distance (Pythagoras).
- **Why the $\max(\cdot, 1)$?** The next formula takes $\log_{10}$ of this. And $\log_{10}(0) = -\infty$, which would blow up. So we refuse to go below 1 metre. It is a guard rail, not physics.

2.2 A.2 Signal strength — the important one

$$\text{RSSI}_{ij} = P_{\text{tx}} - (L_0 + 10n \log_{10} d_{ij}) - \gamma c_{ij}$$

In words: how loud the AP shouts, minus how much the air eats, minus how much the walls eat.

Symbol	Meaning	Our value
RSSI	how strong the signal is when it arrives (dBm; more negative = weaker)	—
P_{tx}	how loud the AP transmits	20 dBm
L_0	how much is lost in the very first metre	40 dB
n	how fast the signal dies with distance	3.3
c_{ij}	how many walls the signal has to pass through	0, 1, 2...
γ	how much <i>each</i> wall costs you	8 dB

Now run it. An AP transmitting at 20 dBm, and a room 10 m away:

- **no walls:** $20 - (40 + 10 \times 3.3 \times \log_{10} 10) = 20 - (40 + 33) = -53$ dBm
- **one wall:** $-53 - 8 = -61$ dBm
- **two walls:** $-61 - 8 = -69$ dBm

Our "usable signal" threshold is $\tau = -66$ dBm. So the *same room at the same distance* is **fine** with one wall in the way and **dead** with two.

Push it further and you get the fact that the whole problem turns on:

An access point reaches 24.8 m through open air — but only 14.2 m through a single concrete wall. One wall costs you 11 metres of range.

That is why AP placement is hard, and why the answer is not "put them in the middle".

And here is the bit that matters for the algorithm. c_{ij} is a *count of walls*. It is 0, or 1, or 2. It is never 1.5. So as you slide an AP smoothly across the floor, the signal in some room drops by **8 dB in one jump** the instant the line of sight crosses a wall. The objective has a **step** in it.

You cannot do calculus on a step. There is no slope to roll down. **That one fact is the entire reason we use a swarm instead of gradient descent**, and if you can say only one thing about Part A's maths, say this.

2.3 A.3 The S-curve (soft coverage)

$$\rho_i = \sigma\left(\frac{\max_j \text{RSSI}_{ij} - \tau}{s}\right), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

In words: each room listens to whichever AP it hears best, and we score it between 0 ("no signal") and 1 ("great signal").

- $\max_j$ — **each room is served by its strongest AP.** It does not matter that the other two are far away.
- σ (the "sigmoid" or logistic) is just an S-shaped squasher. Feed it any number, get back something between 0 and 1. $\sigma(0) = 0.5$ exactly. Big positive input → close to 1. Big negative → close to 0.
- $\tau = -66$ dBm is the pass mark; $s = 4$ dB controls how sharp the S is.

Why not just a hard yes/no? Because a yes/no gives the optimiser no gradient to follow at all — a room is either in or out, and nudging the AP by 10 cm changes nothing until it suddenly flips. The S-curve rewards *margin*: a room with a strong signal scores better than a room that barely scrapes past. That gives the swarm a sense of "warmer / colder".

Running it:

signal	ρ	reading
−53 dBm (10 m, no wall)	0.963	comfortably covered
−61 dBm (10 m, one wall)	0.777	covered, but not luxurious
−66 dBm (exactly at threshold)	0.500	on the knife edge
−69 dBm (10 m, two walls)	0.321	effectively dead

2.4 A.4 Weighted coverage — the score

$$C(\mathbf{x}) = 100 \cdot \frac{\sum_i w_i \rho_i}{\sum_i w_i}$$

In words: the percentage of *students* with signal — not the percentage of rooms.

w_i is how many students live in room i . A 4-person room counts four times as much as a 1-person room. Divide by the total so it lands between 0 and 100.

Tiny example — 3 rooms:

room	students w_i	coverage ρ_i	$w_i \rho_i$
A	4	1.0	4.0
B	1	0.5	0.5
C	2	0.0	0.0
	7		4.5

$C = 100 \times 4.5/7 = \mathbf{64.3\%}$. Two of three rooms have some signal, but only 64% of *students* do, because the dead room has two people in it.

2.5 A.5 The objective (what we maximise)

$$F(\mathbf{x}) = C(\mathbf{x}) - \lambda_{\text{sep}} \sum_{j < k} [\max(0, d_{\text{min}} - \|\mathbf{a}_j - \mathbf{a}_k\|)]^2$$

Scary-looking. It is two things minus each other.

Left half: the coverage score from A.4. We want it big.

Right half: a fine for putting two APs too close together. Read it inside-out:

1. $\|\mathbf{a}_j - \mathbf{a}_k\|$ — how far apart APs j and k are.
2. d_{min} — (that) — how much *closer than allowed* they are. Our d_{min} is 10 m.
3. $\max(0, \dots)$ — **if they're far enough apart, this is zero and there is no fine at all.** The fine only switches on when they're too close.
4. $[\dots]^2$ — square it, so being *very* close is punished much harder than being slightly close.
5. $\sum_{j < k}$ — do that for every *pair* of APs (the $j < k$ just means don't count the same pair twice).
6. $\lambda_{\text{sep}} = 0.30$ — how much we care.

Running it:

APs are...	breach	fine
12 m apart	0	0.00 — legal, no penalty
10 m apart	0	0.00 — exactly on the limit
8 m apart	$2^2 = 4$	1.20
5 m apart	$5^2 = 25$	7.50
stacked on top of each other	$10^2 = 100$	30.00

So piling all three APs in one corner costs you 30 points of fitness. The optimiser learns to spread them out without us ever telling it to.

This is called a **soft penalty**: the rule is not physically impossible to break, it is just expensive. Compare with the floor boundary, which we enforce by **repair** — if a particle wanders off the floor, we just shove it back on.

2.6 A.6 The PSO update

$$\mathbf{v} \leftarrow \underbrace{w\mathbf{v}}_{\text{momentum}} + \underbrace{c_1 r_1 (\mathbf{p}_{\text{best}} - \mathbf{x})}_{\text{my own best}} + \underbrace{c_2 r_2 (\mathbf{g}_{\text{best}} - \mathbf{x})}_{\text{the swarm's best}}, \quad \mathbf{x} \leftarrow \mathbf{x} + \mathbf{v}$$

In words: *keep going the way I was going, plus pull back towards the best spot I've personally found, plus pull towards the best spot anyone has found.*

- $\mathbf{x}$ is one particle — **a complete guess at where all 3 APs go** (6 numbers).
- $\mathbf{v}$ is its velocity: which way it is drifting.
- w is **momentum**, and it shrinks from 0.9 to 0.4 over the run. Early on the particles are careening about (exploring); later they settle (exploiting).
- c_1 pulls it to its own memory; c_2 pulls it to the flock's discovery.
- r_1, r_2 are fresh random numbers each step — the jitter that stops everyone moving in lock-step.

Set $c_2 = 0$ and the third term vanishes. No particle ever hears about anyone else's discovery. That is the ablation, and the result is the whole point of Part A: thirty non-communicating particles score 87.3, which is the same as random guessing.

3 PART B — the water-pump equations

3.1 B.1 The state

$$s = (L, t, g, F)$$

Four numbers that tell you everything you need to know right now: **how full the tank is, what hour it is, is the power on, how much diesel is left.**

$11 \times 24 \times 2 \times 3 = \mathbf{1584}$ possible situations.

The reason this list is exactly right (not shorter, not longer) has a name: the **Markov property**. It means *knowing these four things is enough — the past adds nothing*. If you told me the tank is at 6, it's 3pm, the power is on, and there's one hour of diesel left, I do not need to know what happened yesterday to work out what's likely to happen next.

That property is the *only* reason either algorithm is allowed to work. Both of them assume it.

3.2 B.2 One hour of physics

$$\tilde{L} = \min(L + q, L_{\max}), \quad U = \max(0, D - \tilde{L}), \quad L' = \max(0, \tilde{L} - D)$$

Read left to right, it is just a story:

- $\tilde{L}$ — **pump first**. Add $q = 3$ units. But the tank only holds $L_{\max} = 10$, so min caps it. Anything above spills.
- U — **then the students draw D units**. If they wanted more than the tank had, U is the water they *didn't get*. If the tank had plenty, $\max(0, \cdot)$ makes this 0.
- L' — what's left in the tank afterwards. Never negative.

$\max(0, \cdot)$ appears twice and both times it means the same thing: “*this can't go below zero*”. You can't have negative unmet demand, and you can't have a negative tank.

3.3 B.3 The reward

$$r = -\left(\underbrace{\kappa}_{\text{pump cost}} + \underbrace{c_{\text{spill}} O}_{\text{wasted water}} + \underbrace{c_{\text{short}} U}_{\text{thirsty students}} \right)$$

Everything is negative — it's all cost. The agent's job is to make the total cost as small as possible (i.e. the reward as close to zero as possible).

- grid pump-hour: **1**
- diesel pump-hour: **6** (six times dearer — that's why it's a last resort)
- each unit of water a student wanted and didn't get: **20** ← *by far the worst thing*

That 1-vs-6-vs-20 ratio is the entire personality of the agent. Shortage is so expensive that it will happily burn diesel to avoid it — but diesel is expensive enough that it would much rather have filled the tank earlier on cheap grid power.

3.4 B.4 The subtle one: $R(s, a) = \mathbb{E}_w[r]$

This is the formula in `statement.md` that looks like it's hiding something. It isn't.

$\mathbb{E}[\cdot]$ just means **average**. Demand D is random — some hours 2 students shower, some hours 5. So the reward is random too.

$R(s, a)$ is the **average** reward you'd get if you took action a in state s over and over.

Why it matters: Value Iteration works with the *average* $R(s, a)$ — it has the whole probability table, so it can compute the average directly. Q-learning only ever sees the *actual* reward that actually happened on one particular hour. It never sees the average, because it never sees the probabilities.

That difference **is** model-based versus model-free, written in one line.

3.5 B.5 Discounting — what γ actually does

$$V^\pi(s) = \mathbb{E} \left[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \dots \right]$$

In words: the value of being in a state = all the rewards you'll collect from here on, added up — but the further away a reward is, the less it counts *now*.

With $\gamma = 0.99$ and one step = one hour:

a reward this far away...	...is worth this much today
1 hour	0.990
10 hours	0.904
24 hours (a full day)	0.786
100 hours	0.366

The useful rule of thumb: $1/(1-\gamma)$ **is roughly how far ahead the agent can see**. At $\gamma = 0.99$ that's **100 hours** — comfortably more than the 24-hour cycle the whole problem depends on. That is *why* we picked 0.99 and not 0.9 (which would give only 10 hours of foresight, and the agent would never see the evening outage coming from lunchtime).

Set $\gamma = 0$ and the agent only cares about *this hour*. That's the "myopic" baseline — and it loses by 121%.

3.6 B.6 The Bellman optimality equation

$$V^*(s) = \max_a \left[\underbrace{R(s, a)}_{\text{reward now}} + \gamma \underbrace{\sum_{s'} P(s' | s, a) V^*(s')}_{\text{average value of where I land}} \right]$$

This is the most important formula in Part B and it says something almost obvious:

The best I can do from here = pick the action that maximises (what I get right now + how good the place I land in is, discounted).

Term by term:

- $\max_a$ — try every action, keep the best.
- $R(s, a)$ — the immediate reward, on average.
- $P(s' | s, a)$ — "if I'm in s and do a , what's the chance I end up in s' ?" **This is the model.** It is the thing Value Iteration is handed and Q-learning never sees.
- $\sum_{s'} P(\cdot) V^*(s')$ — average the value of all the places I might land, weighted by how likely each is.

It looks circular — V^* is defined in terms of V^* . It is. The trick is that if you just guess V^* , plug it into the right-hand side, and take the answer as your new guess, you get *closer to the truth every single time*. Repeat until it stops changing. That is **Value Iteration**, and the amount you're wrong shrinks by a factor of γ every sweep — which is exactly what our measured contraction rate of **0.9900** is showing.

3.7 B.7 The Q-learning update

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\underbrace{r + \gamma \max_{a'} Q(s', a') - Q(s, a)}_{\text{how surprised I am}} \right]$$

In words: *I had a guess. Reality gave me a number. Nudge my guess towards reality, a bit.*

- $Q(s, a)$ = my current guess of "how good is doing a in s ".
- $r + \gamma \max_{a'} Q(s', a')$ = what actually happened, plus what I reckon the place I landed is worth. Call this "reality's answer".
- The bracket = reality's answer *minus* my guess. This is the **surprise** (its proper name is the TD error). If I was right, it's zero and nothing changes.
- α = how far to move towards reality. Small α = stubborn, learns slowly but steadily. Big α = jumpy, over-reacts to one unlucky hour.

Note what is NOT in this formula: P . No probabilities anywhere. It only needs s , a , the r that happened, and the s' it landed in. That is what "model-free" means — and it is the whole difference from B.6.

One subtlety worth knowing, because it's a good viva question: that $\max_{a'}$ means the update assumes it will act *optimally* from the next state onwards — even though the ϵ -greedy agent might actually go and do something random. That mismatch is what makes Q-learning **off-policy**, and it's why it can learn the optimal policy while behaving badly the whole time.

3.8 B.8 Regret

$$\text{regret} = 100 \cdot \frac{V^* - V^\pi}{|V^*|}$$

How much worse than perfect, as a percentage. 0% = optimal. 15% = you're losing 15% relative to the best possible.

We can only compute this because our problem is small enough that Value Iteration gives us the *actual* V^* to compare against. On a big problem you'd have no ground truth and would be comparing approximations to approximations.

3.9 The three sentences to take away

1. **Part A's objective has a step in it** (the wall-crossing count is an integer), so it has no gradient, so gradient descent is not an option. Hence a swarm.
2. $P(s' \mid s, a)$ **is the whole story of Part B.** Value Iteration is *given* it and computes the exact answer. Q-learning never sees it and has to feel its way. That is a difference in *what information you have*, not in how clever the algorithm is.
3. **Everything else is bookkeeping** — sums, averages, and "don't go below zero".